

PivotMap — what we built

BTW Buildathon 2026 · Track 2, Graph Intelligence · state at 11:46 IST, 6 September

13
commits on `pivot-work-order`

15
checkpoints across 3 sessions

3
real defects found & fixed

1 → 0
violations, agent-fixed, machine-verified

The idea

A constraint arrives *after* the code is written — a dependency is banned, a vendor is dropped. The developer needs three answers before touching anything: what actually violates this, what breaks if I rip it out, and what can I leave alone. Today that is manual archaeology: `git log` shows what changed but not why, and `grep` finds the banned import but not the three modules built on top of it.

PivotMap is one new command — `entire graph pivot` — that splits the repository into four populations and prints the graph edge behind every single verdict, so a reader can check the claim instead of trusting it.

Verdict	Meaning	Action
INVALIDATED	Has an <code>IMPORTS</code> / <code>CALLS</code> / <code>CONSTRUCTS</code> edge reaching the prohibited dependency	Replace
AT-RISK	Reachable from an invalidated file, breadth-first, capped at depth	Rework + retest
SAFE	No path found to anything invalidated	Leave alone
UNREACHED	Inventory-only language — the parser never checked it	Manual review

No LLM produces a verdict. The interpretive step — a sentence like "we can't send customer data through Library X" becoming the package name — stays *outside* the binary. That is not a

style preference: the plugin's own `doctor` asserts `no_egress=true`, and a model call inside would make its own diagnostic lie.

What we built today

Shipped **File roles** — every file classified `PRODUCTION` | `TEST` | `TEST_FIXTURE` | `GENERATED` | `VENDOR` | `UNKNOWN` from signals that cannot be argued with: a `vendor/` segment, a `testdata|fixtures|mocks` segment, the `_test.go` suffix, or the `Code generated ... DO NOT EDIT.` header read from the file itself.

The role **never changes a verdict** — only how the report is grouped and ranked. 187 at-risk test files that all resolve to one `go test ./internal/sem` were burying the 8 production files somebody actually has to open. The fix is to rank, not to drop: a deleted result is one nobody can audit.

Shipped **--format plan** — emits `pivot-plan/v1`, an agent work order rather than a graph dump. Every file becomes exactly one work item, sorted before ids are assigned so reading top to bottom is reading execution order.

<code>INVALIDATED</code>	<code>-> REPLACE_OR_REMOVE</code>	<code>P0</code> = direct violation in shipped code
<code>AT-RISK</code>	<code>-> VERIFY_AFTER_ROOT_FIX</code>	<code>P1</code> = violation in a test, or shipped code on a bad foundation
<code>SAFE</code>	<code>-> NO_ACTION</code>	<code>P2</code> = covered by a package command, or not a change at all
<code>UNREACHED</code>	<code>-> MANUAL_REVIEW</code>	

Two parts of the schema exist because of how an *agent* fails, not how the graph works: `known_blind_spots` (6) and `execution_contract.must_not_claim` (6).

Shipped **--exclude-tests**, **applied before classification** — not an output filter. A production file whose only route to invalidated code runs through a test comes back `SAFE`, because a test is not a foundation anything ships on. The excluded count is printed, because a file that was never classified is not a file that came back `Safe`.

The verification loop — the centrepiece

Run 1 `pivot --format plan -->` bounded work order (1 `P0`, 4 `P1`) | fresh agent session, no prior context executes `P0` only, commits | Run 2 `pivot --format plan -->` re-derived from

source. count must drop.

Run live against **gorilla/mux** with `--dependency bytes`. mux was the target because the constraint has to be *satisfiable* — banning `os/exec` in our own repo, or `net/http` in a router, asks an agent to delete the reason the code exists.

metric	before db9d1d0	after 31bca9f
invalidated	1	0
at-risk	4	0
P0 / P1	1 / 4	0 / 0

The count alone proves nothing — `invalidated == 0` is equally satisfied by deleting the code. So the diff is the other half of the evidence:

```
regexp.go | 15 ++++++----- 1 file changed, 7 insertions(+), 8 deletions(-)
M regexp.go                  no test file touched
go test ./... -> ok          green before AND after
```

A cold agent replaced `bytes.IndexAny/IndexByte` with the `strings` equivalents and preserved behaviour. **No LLM graded the result** — most agent demos have agents grading each other.

Three defects found today

1. Evidence cited against the wrong file fixed · 0fa4a48

```
before  evidence: CALLS -> regexp.go:676      # regexp.go is 413 lines. does not exist.
after   evidence: CALLS route.go:676 -> regexp.go # 676 is in route.go (828 lines). correct.
```

The verdict was never wrong — `route.go:259` really does call `newRouteRegexp`. Only the citation was.

Found by the executing agent, not by a test. Its brief told it to read the cited source before editing, and it reported the line was not there. The plan instructed an agent to distrust the plan, and that is what surfaced the bug.

2. A test read ambient `FORCE_COLOR` fixed · 3170d58

It compared against an exact plain string, but the headline passes through `ShouldUseColor`, which honours `FORCE_COLOR` before checking for a terminal. CI sets no such variable and passes; a developer whose shell exports it sees a failure that looks real. Now pinned, verified both ways.

3. BUILDATHON.md printed an unreproducible plan id fixed · cf70f83

`plan_id` is a digest over head commit + constraint + depth + flags + counts, so it is *built* to change with every commit. Printing one in prose guarantees staleness.

The audit's replacement id was **not** substituted — this session did not independently reproduce it, and swapping an unverified number for an unverified number is not a fix. The claim was removed instead.

Real numbers — this repository

Frozen snapshot at `934d180`, 634 files: 277 PRODUCTION, 266 TEST, 91 TEST_FIXTURE.

Run	INVAL	AT-RISK	SAFE	UNREACHED	excluded
<code>net/http</code>	3	78	468	85	0
<code>net/http --exclude-tests</code>	0	0	204	73	357
<code>os/exec</code>	32	287	230	85	0
<code>os/exec --exclude-tests</code>	8	100	96	73	357
<code>os/exec --exclude-tests --depth 1</code>	8	11	185	73	357

`net/http` is test-only here. All three importers are test files or a fixture — the constraint would not touch a shipped line. That clean zero is the report working, not failing, and it is only legible because the excluded count is printed beside it.

`os/exec` is the real one. 8 production files — six are the git-subprocess and verification layers, two are benchmark tooling. At `--depth 1` that plus 11 direct dependents is a 19-file shortlist, down from 319: a **94% reduction** with nothing that ships lost.

The honest limit — stated, not hidden

SAFE means "no path was found in this local snapshot". It is not proof of non-use. Calls through interfaces, reflection, or generated code leave no edge to follow. Observed concretely: `impact --symbol Route.Match` on gorilla/mux reports **0 callers** even though `Router.Match` calls it through an interface. The `impact` command also self-reports `Completeness: degraded` for `Go` next to its own answer — a tool naming its own incompleteness. UNREACHED exists as a separate verdict for exactly this reason.

Where it lives

Thing	Where
Branch (all work)	<code>pivot-work-order</code> — 13 commits, pushed, in sync
<code>main</code>	<code>protected</code> stale at <code>3a2a715</code> ; push rejected by the mirror. Judges must be pointed at the branch.
Source	<code>internal/cli/pivot.go</code> , <code>pivot_roles.go</code> , <code>pivot_plan.go</code>
Graph evidence	<code>docs/demo/graph-evidence.txt</code> — search, impact, semantic diff
Loop evidence	<code>docs/demo/mux-before.json</code> , <code>mux-after.json</code>
Demo run sheet	<code>docs/demo/DEMO.md</code>
Agent brief	<code>.entire/pivot-agent-brief.md</code>